

Task 2 - QA Strategy & Thinking

1. Where do you start?

Due to the urgency and lack of QA infrastructure, my immediate priority will be risk assessment and establishing a rapid, pragmatic QA foundation. I would do the following:

- Understand the product: Meet with product managers and developers to grasp the app's core features, user flows, and its most critical use cases—especially those involving real user funds.
- Go through tech documentation, Figma, business requirements docs etc.
- Identify the most critical paths: Map out the highest-impact user journeys (e.g., Account login, portfolio view, fund deposit/withdrawal, trade execution) where failure would have severe consequences.
- Assess compliance and security obligations: Since this is a fintech app, I would clarify regulatory, compliance, and security requirements with stakeholders.
- Review current release process: Understand how code is shipped, what (if any) checks exist, and where the greatest risks of regression or failure lie.

2. How would you approach testing this app?

With only two weeks until release and no test suite, I would implement a risk-based, high-leverage approach:

1. Create Test Plan and Strategy
2. Manual exploratory testing: Begin immediate manual testing of the most critical flows (e.g., authentication, trading, funds movement) and high-level test cases for repeatability.
3. Smoke tests: Define and execute a minimal set of smoke tests to ensure basic app operability on both iOS and Android.
4. Bug triage and reporting: Hold regular bug triage sessions with developers and product stakeholders to prioritise issues based on business impact, customer experience, and release risk. Focus on resolving critical and high-severity defects before release.
5. Test case documentation: Start capturing test cases for critical and high-risk areas, even if only in a shared document or spreadsheet at first.
6. Collaborate with the team: Work closely with developers through feature demos, requirement discussions, and pair testing where appropriate..

3. What does QA look like inside a sprint, from ticket creation through to regression?

Within a sprint, QA should be integrated from the very beginning:

1. Ticket creation: Ensure tickets include clear acceptance criteria and, where possible, security and compliance notes. Collaborate with product and dev to clarify requirements and risks up front.
2. Early involvement: Review user stories and designs for testability and risk, raising concerns before development begins.

3. Continuous testing: Test new features as soon as builds are available, using exploratory and scripted tests. Provide fast feedback to developers to avoid late-stage surprises.
4. Bug tracking and resolution: Log, prioritise, and track issues. Retest fixes promptly and ensure regression risk is managed as the sprint progresses.
5. Pre-release regression: Before merging or releasing, run a regression suite (manual or automated, as available) to check for unintended side effects.
6. Few additional steps can be added in the future like developer code review by QA, making unit test coverage is passing in the build checks etc. In my current company Binance we review the code and make sure the checks are passing before starting QA testing

4. What does your ideal regression suite look like?

An ideal regression suite for a fintech mobile app would be:

- Automated and repeatable: Covers all critical paths, automated for both iOS and Android, and runs on every build or before every release.
- Risk-focused: Prioritises tests for flows where onboarding, user funds, data integrity, and compliance are at stake (e.g., login, trade execution, deposit/withdrawal, transaction history).
- Scalable and maintainable: Designed to grow with the application, using modular and reusable test components that are easy to update.
- Fast feedback: Runs in minutes, not hours, so issues are caught early and the release process isn't blocked.
- Data-independent: Uses controlled test data and environment setup so tests can run repeatedly without manual intervention.
- Living documentation: Test cases are maintained alongside code, easily accessible, and understood by the whole team.
- Reporting: Produces clear reports with screenshots, logs, traces, and failure details so developers can quickly identify and resolve issues.

5. What would keep you up at night about this app specifically and releasing to the public ?

I will have following concerns :

- A bug that put user funds at risk
- Security bugs: Exposure of personal or financial data, insecure authentication could lead to breaches, fraud, or regulatory penalties.
- Compliance violations: Not saving user's data during onboarding, missing required audit trails, KYC/AML checks etc.
- Uncaught regressions: With fast shipping and no test suite, there's a high risk that new features break existing ones, especially in critical paths.
- Poor UI experience or High Latency: Bugs leading to failed trades, unresponsive UI, or crashes—especially on one platform—could erode trust quickly.
- Lack of monitoring and rollback: If the app is released without proper monitoring/logging, issues may go undetected